

Working with CSV & JSON in Python

A practical guide to reading, writing, and transforming structured data

CSV and JSON are the two most common formats you'll encounter when moving data in and out of Python programs. CSV (comma-separated values) is a simple, spreadsheet-like row-and-column format. JSON (JavaScript Object Notation) is a flexible, nested format built from key-value pairs, lists, and primitive values. This guide walks through Python's built-in tools for both, plus the pandas library for larger or messier datasets, and shows common pitfalls to avoid.

Aspect	CSV	JSON
Shape	Flat rows & columns	Nested objects & arrays
Best for	Tabular / spreadsheet data	Hierarchical / API data
Types	Everything is text by default	Numbers, strings, booleans, null, nested objects
Python module	csv	json
Typical source	Excel, databases, exports	Web APIs, config files, NoSQL

Part 1 — Working with CSV Files

1.1 Reading a CSV file

The built-in `csv` module handles quoting, delimiters, and edge cases you don't want to reinvent by hand. Two common readers: `reader` (returns lists) and `DictReader` (returns dictionaries keyed by header).

```
import csv

with open("data.csv", newline="", encoding="utf-8") as f:
    reader = csv.reader(f)
    header = next(reader) # first row = column names
    for row in reader:
        print(row) # row is a list of strings
```

`DictReader` is usually more convenient because you can refer to columns by name instead of numeric position:

```
import csv

with open("data.csv", newline="", encoding="utf-8") as f:
    reader = csv.DictReader(f)
    for row in reader:
        print(row["name"], row["email"])
```

■ Always pass `newline=""`

When opening a CSV file for reading or writing, pass `newline=""` to `open()`. Without it, extra blank lines can appear on Windows because of how line endings get translated.

1.2 Writing a CSV file

```
import csv

rows = [
    {"name": "Amina", "email": "amina@example.com"},
    {"name": "Karim", "email": "karim@example.com"},
]

with open("output.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(f, fieldnames=["name", "email"])
    writer.writeheader()
    writer.writerows(rows)
```

1.3 Custom delimiters and dialects

Not every file uses commas. Semicolons and tabs are common in European exports or TSV files. Specify the delimiter explicitly:

```
with open("data_semicolon.csv", newline="", encoding="utf-8") as f:
    reader = csv.reader(f, delimiter=";")
    for row in reader:
        print(row)

# Tab-separated
with open("data.tsv", newline="", encoding="utf-8") as f:
    reader = csv.reader(f, delimiter="\t")
```

1.4 Type conversion

Every value read from a CSV file is a string, even numbers and dates. You must convert types explicitly:

```
import csv
from datetime import datetime

with open("sales.csv", newline="", encoding="utf-8") as f:
    reader = csv.DictReader(f)
    for row in reader:
        amount = float(row["amount"])
        date = datetime.strptime(row["date"], "%Y-%m-%d")
        print(date, amount)
```

1.5 CSV with pandas (recommended for larger datasets)

For anything beyond a quick script — filtering, aggregating, merging, or handling messy real-world files — pandas is the standard tool. It infers types automatically and offers a huge set of options.

```
import pandas as pd

df = pd.read_csv("data.csv") # reads into a DataFrame
print(df.head()) # preview first 5 rows
print(df.dtypes) # inferred column types

df["amount"] = df["amount"].astype(float)
df_filtered = df[df["amount"] > 100]

df_filtered.to_csv("filtered.csv", index=False)
```

■ Handling messy CSVs with pandas

Useful `read_csv` arguments: `sep=";"` for a custom delimiter, `encoding="latin-1"` for non-UTF-8 files, `na_values=["NA", "-"]` to treat extra tokens as missing, and `dtype={"id": str}` to force a column to stay as text (e.g. zip codes or IDs that shouldn't lose leading zeros).

Part 2 — Working with JSON Files

2.1 Reading JSON

The built-in `json` module converts JSON text into native Python objects — dicts, lists, strings, numbers, booleans, and `None`.

```
import json

with open("data.json", encoding="utf-8") as f:
    data = json.load(f) # parses file directly into a Python object

print(data["name"])
print(data["tags"][0])
```

If the JSON is already a string in memory (e.g. from an API response), use `json.loads()` instead:

```
import json

json_text = '{"name": "Amina", "age": 29, "active": true}'
data = json.loads(json_text)
print(data["name"], data["age"])
```

2.2 Writing JSON

```
import json

data = {
    "name": "Amina",
    "age": 29,
    "skills": ["python", "sql", "docker"],
    "active": True,
}

with open("output.json", "w", encoding="utf-8") as f:
    json.dump(data, f, indent=2, ensure_ascii=False)

# Or as a string
json_text = json.dumps(data, indent=2)
print(json_text)
```

■ indent and ensure_ascii

`indent=2` pretty-prints the output for readability. `ensure_ascii=False` keeps non-ASCII characters (accents, Arabic, emoji, etc.) readable instead of escaping them to `\uXXXX` sequences.

2.3 Type mapping between JSON and Python

JSON	Python
object	dict
array	list
string	str
number (int)	int
number (float)	float
true / false	True / False
null	None

2.4 Nested data and safe access

JSON often nests deeply. Use `.get()` to avoid crashing on missing keys, and handle lists of objects with normal iteration:

```
data = {
    "user": {"name": "Karim", "address": {"city": "Cairo"}},
    "orders": [
        {"id": 1, "total": 42.5},
        {"id": 2, "total": 17.0},
    ],
}

city = data.get("user", {}).get("address", {}).get("city", "Unknown")

order_total = sum(order["total"] for order in data["orders"])
print(city, order_total)
```

2.5 Custom objects and dates

JSON has no native date type, and it can't serialize arbitrary Python objects directly. Use a custom encoder or convert values to strings first:

```
import json
from datetime import datetime

def default_serializer(obj):
    if isinstance(obj, datetime):
        return obj.isoformat()
    raise TypeError(f"Type {type(obj)} not serializable")

data = {"created": datetime.now()}
json_text = json.dumps(data, default=default_serializer)
```

2.6 JSON with pandas

```
import pandas as pd

df = pd.read_json("data.json") # flat/simple JSON
df.to_json("output.json", orient="records", indent=2)

# Deeply nested JSON: flatten it first
import json
with open("nested.json") as f:
    raw = json.load(f)
df = pd.json_normalize(raw, record_path="orders", meta=["user_id"])
```

Part 3 — Converting Between CSV and JSON

3.1 CSV → JSON

```
import csv
import json

with open("data.csv", newline="", encoding="utf-8") as f:
    reader = csv.DictReader(f)
    rows = list(reader)

with open("data.json", "w", encoding="utf-8") as f:
    json.dump(rows, f, indent=2)
```

3.2 JSON → CSV

This only works cleanly when the JSON is a flat list of objects with the same keys. Nested or irregular JSON needs flattening first (e.g. with `pandas.json_normalize`).

```
import json
import csv

with open("data.json", encoding="utf-8") as f:
    data = json.load(f) # a list of dicts

fieldnames = data[0].keys()
with open("data.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(f, fieldnames=fieldnames)
    writer.writeheader()
    writer.writerows(data)
```

3.3 One-liner with pandas

```
import pandas as pd

pd.read_csv("data.csv").to_json("data.json", orient="records", indent=2)
pd.read_json("data.json").to_csv("data.csv", index=False)
```

Part 4 — Common Pitfalls

- **Encoding errors:** always specify `encoding="utf-8"` when opening files, especially with non-English text.
- **Everything is a string in CSV:** numbers, booleans, and dates need explicit conversion after reading.
- **Trailing commas in JSON:** unlike Python dicts, JSON does not allow a trailing comma before a closing brace or bracket — it will raise a parse error.
- **Single vs double quotes:** JSON requires double quotes for strings and keys; Python dict literals with single quotes are not valid JSON.

- **Large files:** for CSV files too big for memory, iterate row-by-row instead of loading it all, or use `pandas.read_csv(..., chunksize=10000)`. For huge JSON, consider a streaming parser like `ijson`.
- **Missing keys:** use `dict.get(key, default)` for JSON and check for empty strings in CSV rather than assuming a field always exists.

Task	Best tool
Quick script, small file	built-in csv / json modules
Filtering, joining, aggregating	pandas
Deeply nested JSON → table	<code>pandas.json_normalize</code>
Huge files that don't fit in memory	chunked <code>pandas.read_csv</code> , or <code>ijson</code> for JSON
API responses	<code>requests</code> + <code>response.json()</code>